



Why learn with us ?



Training From Experts



One - on – One Guidance



100% Practical



Real-Time Projects & Case Studies



Placement Preparation Support



OFFLINE CLASSES



ONLINE LECTURE



RECORDED ACCESS

Contact :- 8085896740

 @ hello_world_institute

 Hello World Institute

Address : 2nd Floor , Near KFC , Tower Square , Bhawarkua , Indore

Files

Sometimes, it is not enough to only display the data on the console. The data to be displayed may be very large, and only a limited amount of data can be displayed on the console since the memory is volatile, it is impossible to recover the programmatically generated data again and again.

The file handling plays an important role when the data needs to be stored permanently into the file. A file is a named location on disk to store related information. We can access the stored information (non-volatile) after the program termination.

The file-handling implementation is slightly lengthy or complicated in the other programming language, but it is easier and shorter in Python. **Types of File**

There are two types of files in Python and each of them are explained below in detail with examples for your easy understanding.

They are:

- Binary file
- Text file

Python File Handling Operations

Most importantly there are 4 types of operations that can be handled by Python on files:

- Open
- Read
- Write
- Close

Other operations include:

- Rename
- Delete

Python Create and Open a File

Python has an in-built function called `open()` to open a file.



OFFLINE CLASSES



ONLINE LECTURE



RECORDED ACCESS

HELLO WORLD INSTITUTE

It takes a minimum of one argument as mentioned in the below syntax. The open method returns a file object which is used to access the write, read and other inbuilt methods.

Syntax:

```
file_object = open(file_name, mode)
```

The mode in the open function syntax will tell Python as what operation you want to do on a file.

- **'r' – Read Mode:** Read mode is used only to read data from the file.
- **'w' – Write Mode:** This mode is used when you want to write data into the file or modify it. Remember write mode overwrites the data present in the file.
- **'a' – Append Mode:** Append mode is used to append data to the file. Remember data will be appended at the end of the file pointer.
- **'r+' – Read or Write Mode:** This mode is used when we want to write or read the data from the same file.
- **'a+' – Append or Read Mode:** This mode is used when we want to read data from the file or append the data into the same file.

Note: The above-mentioned modes are for opening, reading or writing text files only.

While using binary files, we have to use the same modes with the letter **'b'** at the end. So that Python can understand that we are interacting with binary files.

- **'wb' –** Open a file for write only mode in the binary format.
- **'rb' –** Open a file for the read-only mode in the binary format.
- **'ab' –** Open a file for appending only mode in the binary format.
- **'rb+' –** Open a file for read and write only mode in the binary format.
- **'ab+' –** Open a file for appending and read-only mode in the binary format.

Example

```
#opens the file file.txt in read mode
fileptr = open("file.txt","r")

if fileptr:
    print("file is opened successfully")
```



OFFLINE CLASSES



ONLINE LECTURE



RECORDED ACCESS

Contact :- 8085896740



@ hello_world_institute



Hello World Institute

Address : 2nd Floor , Near KFC , Tower Square , Bhawarkua , Indore

In the above code, we have passed **filename** as a first argument and opened file in read mode as we mentioned **r** as the second argument. The **fileptr** holds the file object and if the file is opened successfully, it will execute the print statement

The close() method

Once all the operations are done on the file, we must close it through our Python script using the **close()** method. Any unwritten information gets destroyed once the **close()** method is called on a file object.

We can perform any operation on the file externally using the file system which is the currently opened in Python; hence it is good practice to close the file once all the operations are done.

Syntax:

```
fileobject.close()
```

Python Write to File

In order to write data into a file, we must open the file in write mode. We need to be very careful while writing data into the file as it overwrites the content present inside the file that you are writing, and all the previous data will be erased.

We have two methods for writing data into a file as shown below.

- write(string)
- writelines(list)

Example 1:

```
my_file = open("test.txt", "w")  
my_file.write("Hello World")
```

Example 3:

```
fruits = ["Apple\n", "Orange\n", "Grapes\n", "Watermelon"]  
my_file = open("test.txt", "w")  
my_file.writelines(fruits)
```



OFFLINE CLASSES



ONLINE LECTURE



RECORDED ACCESS

The above code writes a **list of data** into the 'test.txt' file simultaneously.

Python Append to File

To append data into a file we must open the file in 'a+' mode so that we will have access to both the append as well as write modes.

Example 1:

```
my_file = open("test.txt", "a+")  
my_file.write ("Strawberry")
```

The above code appends the string 'Apple' at the **end** of the 'test.txt' file.

Example 3:

```
fruits = ["\nBanana", "\nAvocado", "\nFigs", "\nMango"]  
my_file = open("test.txt", "a+")  
my_file.writelines(fruits)
```

The above code **appends a list of data** into a 'test.txt' file.

Python Read From File

In order to read a file in python, we must open the file in read mode. **There are three ways in which we can read the files in python.**

- read()
- read([n])
- readlines()

Here, n is the number of bytes to be read.



OFFLINE CLASSES



ONLINE LECTURE



RECORDED ACCESS

Example 1:

```
my_file = open("test.txt", "r")  
print(my_file.read(5))
```

Example 2:

```
my_file = open("test.txt", "r")  
print(my_file.read())
```

Example 3:

```
my_file = open("test.txt", "r")  
print(my_file.readline(2))
```

Writing and Reading Data from a Binary File

Binary files store data in the binary format (0's and 1's) which is understandable by the machine. So when we open the binary file in our machine, it decodes the data and displays in a human-readable format.

Pickle Module, Dump() & Load() in Binary Files in Python :

Pickling: The process of converting the structure (lists and dictionary etc.) into a byte stream just before writing to the file. This is also called as object serialization.



OFFLINE CLASSES



ONLINE LECTURE



RECORDED ACCESS

`dump()` function: We use `dump()` method to perform pickling operation on our **Binary Files**. It returns the object representation in byte mode. The `dump()` method belongs to `pickle` module.

Syntax:

```
#pickling_in_python import pickle  
pickle.dump(object,file)
```

Example 1:

```
#Example_pickling_in_python  
  
import pickle  
  
def write():  
    file = open("binary.dat",'wb')  
    x = [1,2,3,4,5] #data we wrote in file  
    pickle.dump(x,file)  
    file.close()  
  
write()
```

Unpickling: The reverse conversion of byte stream back to the structure (lists, dictionary, tuples etc.) refers to unpickling. Basically it is the reverse operation of pickling. This is also called de-serialization. We use `load()` method for unpickling. This `load()` function/method also belongs to `pickle` module.



OFFLINE CLASSES



ONLINE LECTURE



RECORDED ACCESS

load() function: In pickle module, load() function is used to read data from a binary file or file object.

Syntax:

```
#Syntax_unpickling_in_python
```

```
import pickle
```

```
pickle.load(file)
```

Example 2:

```
#Example_unpickling
```

```
import pickle
```

```
def read():
```

```
    file = open("binary.dat", 'rb')
```

```
    data = pickle.load(file)
```

```
    file.close()
```

```
    print(data)
```

```
read()
```

Output:

```
[1, 2, 3, 4, 5]
```

Python File Methods



OFFLINE CLASSES



ONLINE LECTURE



RECORDED ACCESS

Function	Explanation
open()	To open a file
close()	Close an open file
fileno()	Returns an integer number of the file
Function	Explanation
read(n)	Reads 'n' characters from the file till end of the file
readable()	Returns true if the file is readable
readline()	Read and return one line from the file
readlines()	Reads and returns all the lines from the file
seek(offset)	Change the cursor position by bytes as specified by the offset
seekable()	Returns true if the file supports random access
tell()	Returns the current file location
writable()	Returns true if the file is writable
write()	Writes a string of data to the file
writelines()	Writes a list of data to the file



OFFLINE CLASSES



ONLINE LECTURE



RECORDED ACCESS